Manual Explicativo Completo do Código

Disciplina: Sistemas Operacionais | **Professor:** Flávio Giraldeli
Tema: Introdução à Programação Multithread com PThreads (Busca Concorrente de Números Primos)

> **Objetivo deste documento**
>
> Este material explica detalhadamente, bloco por bloco e linha por linha, como o código do arquivo `main.c` funciona. Foi elaborado com o raciocínio claro e direto de um estudante de Sistemas Operacionais, fornecendo toda a bagagem necessária para compreender o programa e responder a qualquer pergunta em uma entrevista presencial.

1. Visão Geral do Problema

O trabalho consiste em receber uma matriz de números naturais pseudoaleatórios (entre 0 e 31999) gerada com semente fixa e contar quantos números primos existem nela, comparando duas abordagens:

1. **Busca Serial:** Uma única thread percorre toda a matriz elemento por elemento sequencialmente. Esse tempo serve como linha de base (referência).
2. **Busca Paralela:** Múltiplas threads dividem a tarefa trabalhando em "macroblocos" (submatrizes). A thread que termina seu macrobloco busca imediatamente o próximo macrobloco livre na fila global compartilhada.

2. As Bibliotecas Incluídas

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
```

- `pthread.h`: Funções para criação, manipulação e sincronização de threads POSIX (`pthread_create`, `pthread_join`, `pthread_mutex_t`).
- `stdio.h`: Funções de entrada e saída padrão (`printf`).
- `stdlib.h`: Alocação dinâmica de memória (`malloc`, `free`) e gerador de números aleatórios (`rand`, `srand`).
- `math.h`: Função `sqrt()` para calcular a raiz quadrada na verificação de números primos.
- `time.h`: Função padrão `clock()` e constante `CLOCKS_PER_SEC` para medição do tempo de execução.

3. Configurações Parametrizadas (#define)

```
#define MATRIZ_LINHAS 1000
#define MATRIZ_COLUNAS 1000
#define BLOCO_LINHAS 100
#define BLOCO_COLUNAS 100
#define NUM_THREADS 4
#define SEMENTE 42
```

- **Por que usar `#define`?** O professor exige parametrização para poder testar com facilidade diferentes tamanhos de matriz (ex: 10000x10000) e diferentes dimensões de macroblocos (de 1x1 até a matriz inteira) sem alterar o código das funções.
- **`SEMENTE 42`:** Garante reprodutibilidade. Como `srand(42)` é fixo, a matriz conterá exatamente os mesmos números em todas as execuções, permitindo comparar de forma justa o tempo serial com o tempo paralelo.

4. Variáveis Globais e Sincronização

```
int **matriz = NULL;
int total_primos_paralelo = 0;
int proximo_macrobloco = 0;
int total_macroblocos = 0;

pthread_mutex_t mutex_bloco;
pthread_mutex_t mutex_primos;
```

O professor exigiu no PDF que a matriz e as variáveis de controle fossem **globais e compartilhadas**:

- `matriz`: Ponteiro para ponteiro (`int**`) que apontará para a matriz alocada dinamicamente no heap.
- `total_primos_paralelo`: Variável acumuladora onde todas as threads somam os primos que encontram.
- `proximo_macrobloco`: Atua como uma "senha de balcão". Começa em 0. Cada thread que quer trabalhar pega o valor atual e incrementa esta variável.
- `mutex_bloco`: Exclusão mútua que impede que duas threads peguem o mesmo macrobloco simultaneamente.
- `mutex_primos`: Exclusão mútua que impede que duas threads alterem `total_primos_paralelo` ao mesmo instante, evitando condição de corrida (Race Condition).

5. Verificação de Números Primos: A Função `ehPrimo`

```
int ehPrimo(int n)
{
    if (n < 2)
    {
        return 0;
    }

    int squirtle = sqrt(n);
    for (int i = 2; i ≤ squirtle; i++)
    {
        if (n % i == 0)
        {
            return 0;
        }
    }
    return 1;
}
```

O Raciocínio por trás desta função:

1. **Caso base (`n < 2`):** 0 e 1 não são primos por definição matemática.
2. **O cálculo de `squirtle = sqrt(n)` fora do loop:** Matematicamente, se um número n tem um divisor composto, pelo menos um dos fatores é menor ou igual à sua raiz quadrada. Calcular a raiz uma única vez fora do loop economiza milhões de operações desnecessárias.
3. **Interrupção imediata:** Se `n % i == 0`, significa que encontramos um divisor; portanto, o número não é primo e a função retorna `0` imediatamente.

6. Medição Tradicional de Tempo com `clock()`

```
clock_t inicio_serial = clock();
int primos_serial = buscaSerial();
clock_t fim_serial = clock();
double tempo_serial = (double)(fim_serial - inicio_serial) / CLOCKS_PER_SEC;
```

Utiliza a função canônica `clock()` da biblioteca padrão `<time.h>`. Ela registra o instante de início e fim da busca, dividindo a diferença pela constante `CLOCKS_PER_SEC` para obter o tempo em segundos.

7. Gerenciamento de Memória Dinâmica

A. Alocação (`alocarMatriz`)

```
void alocarMatriz()
{
    matriz = (int **)malloc(MATRIZ_LINHAS * sizeof(int *));
    if (matriz == NULL) exit(1);

    for (int i = 0; i < MATRIZ_LINHAS; i++)
    {
        matriz[i] = (int *)malloc(MATRIZ_COLUNAS * sizeof(int));
        if (matriz[i] == NULL) exit(1);
    }
}
```

B. Preenchimento e Liberação

```
 void preencherMatriz()
{
    srand(SEMENTE);
    for (int i = 0; i < MATRIZ_LINHAS; i++)
    {
        for (int j = 0; j < MATRIZ_COLUNAS; j++)
        {
            matriz[i][j] = rand() % 32000;
        }
    }
}

void liberarMatriz()
{
    if (matriz ≠ NULL)
    {
        for (int i = 0; i < MATRIZ_LINHAS; i++) free(matriz[i]);
        free(matriz);
        matriz = NULL;
    }
}
```

8. A Busca Paralela e as Threads

A. A Rotina da Thread (`funcaoThread`)

```
void *funcaoThread(void *arg)
{
    int id = *(int *)arg;
    int blocos_por_linha = MATRIZ_COLUNAS / BLOCO_COLUNAS;
    int blocos_processados = 0;

    while (1)
    {
        int meu_bloco = -1;

        pthread_mutex_lock(&mutex_bloco);
        if (proximo_macrobloco < total_macroblocos)
        {
            meu_bloco = proximo_macrobloco;
            proximo_macrobloco++;
        }
        pthread_mutex_unlock(&mutex_bloco);

        if (meu_bloco == -1) break;

        blocos_processados++;

        int bloco_lin = meu_bloco / blocos_por_linha;
        int bloco_col = meu_bloco % blocos_por_linha;

        int lin_inicio = bloco_lin * BLOCO_LINHAS;
        int lin_fim = lin_inicio + BLOCO_LINHAS;

        int col_inicio = bloco_col * BLOCO_COLUNAS;
        int col_fim = col_inicio + BLOCO_COLUNAS;

        int primos_deste_bloco = 0;
        for (int i = lin_inicio; i < lin_fim; i++)
        {
            for (int j = col_inicio; j < col_fim; j++)
            {
                if (ehPrimo(matriz[i][j]))
                {
                    primos_deste_bloco++;
                }
            }
        }
```

```
        pthread_mutex_lock(&mutex_primos);
        total_primos_paralelo += primos_deste_bloco;
        pthread_mutex_unlock(&mutex_primos);
    }
    return NULL;
}
```

Pontos Cruciais para Entender e Defender:

1. **O parâmetro `arg`:** Passamos o ID da thread (`int id = *(int *)arg;`) para que cada thread saiba seu índice numérico e possa contabilizar quantos blocos processou.
2. **A Seção Crítica 1 (Fila Dinâmica):** Garante que cada macrobloco seja processado por exatamente uma thread.
3. **Cálculo $O(1)$ das coordenadas:** Convertemos o índice linear `meu_bloco` em coordenadas de início e fim da submatriz usando divisão inteira e resto, sem gastar 1 byte de RAM extra.
4. **A Seção Crítica 2 (Soma Global):** Cada thread conta os primos do bloco localmente em `primos_deste_bloco` e só trava o `mutex_primos` uma vez ao final de cada bloco.

9. Perguntas Frequentes do Professor na Entrevista

P: Por que você usou dois mutexes diferentes em vez de usar apenas um?

R: Porque são seções críticas independentes! O `mutex_bloco` cuida da distribuição de tarefas, enquanto o `mutex_primos` cuida da soma do total de primos. Se usássemos apenas um mutex para tudo, uma thread que estivesse apenas somando primos iria bloquear outra thread que só queria pegar uma nova tarefa, gerando contenção desnecessária e reduzindo o desempenho.

P: O que acontece se removermos os mutexes?

R: Ocorrerão Condições de Corrida (Race Conditions). Múltiplas threads lerão a variável global de primos com o mesmo valor antigo e sobrescreverão o valor umas das outras (perda de atualizações). Como resultado, a contagem final paralela será **menor** do que a contagem serial correta. Isso fica mais grave quanto menores forem os macroblocos, pois mais disputas simultâneas acontecem.

P: O que é Speedup e por que com 4 threads ele não atinge exatamente 4.0x?

R: O Speedup é a razão entre o tempo serial e o tempo paralelo: $S = \frac{T_{serial}}{T_{paralelo}}$. Ele não chega a 4.0x perfeito devido a fatores explicados pela Lei de Amdahl: existe uma parte do código que é puramente serial (como inicializar variáveis e criar as threads), além do custo operacional do Sistema Operacional (overhead de criação de threads, trocas de contexto e espera nos mutexes).